

Freeside — Complete Bachelor's Implementation Guide

Revised: Supabase · Onboarding Flow · Hybrid Energy Intelligence

What You're Building

A focused, academically rigorous proof-of-concept with six parts:

- | | |
|------------------------|---|
| 0. Project Foundation | → Supabase + Next.js + FastAPI |
| 1. Onboarding Flow | → Who are you? Goals? Connect Calendar? |
| 2. Energy Intelligence | → Google Calendar → AI inference → user confirms |
| 3. Smart Task Routing | → Cognitive load algorithm driven by energy score |
| 4. AI Co-Pilot | → Context-aware assistant + micro-step generation |
| 5. Gamification | → XP, progress bar, completion feedback |
| 6. Analytics Layer | → Data capture for your thesis evidence |

PART 0 — Project Foundation

Step 1 — Create your Supabase project

1. Go to supabase.com → New Project
2. Name it "freeside" → choose a strong DB password → save it somewhere
3. Wait 2 minutes for it to provision
4. Go to Settings → API → copy your `Project URL` and `anon public key`

Supabase gives you: hosted PostgreSQL, Auth (email/password + OAuth), a visual table editor, and real-time subscriptions — all free.

Step 2 — Build your database schema in Supabase

Go to the Supabase Dashboard → SQL Editor → paste and run this:

```

-- USERS PROFILE (extends Supabase's built-in auth.users)
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) PRIMARY KEY,
  name TEXT,
  role TEXT,                                -- 'student', 'professional', 'entrepreneur'
  productive_day_description TEXT,          -- "What does a productive day look like?"
  peak_focus_time TEXT,                    -- 'morning', 'afternoon', 'evening'
  daily_work_hours INTEGER,
  work_style TEXT,                        -- 'long_blocks', 'short_sprints'
  google_calendar_connected BOOLEAN DEFAULT FALSE,
  google_refresh_token TEXT,              -- stored encrypted
  xp_total INTEGER DEFAULT 0,
  onboarding_completed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- GOALS (user's 1-3 big objectives)
CREATE TABLE public.goals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id),
  title TEXT NOT NULL,                    -- "Launch my side project"
  category TEXT,                          -- 'work', 'personal', 'health', 'learning'
  timeframe TEXT,                         -- '1 month', '3 months', '6 months'
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- TASKS
CREATE TABLE public.tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id),
  goal_id UUID REFERENCES public.goals(id), -- optional link to a goal
  title TEXT NOT NULL,
  description TEXT,
  cognitive_load_score INTEGER CHECK (cognitive_load_score BETWEEN 1 AND 10),
  status TEXT DEFAULT 'pending',          -- 'pending', 'active', 'completed', 'rerouted'
  source TEXT DEFAULT 'manual',           -- 'manual', 'ai_generated' (micro-steps)
  parent_task_id UUID REFERENCES public.tasks(id), -- for micro-steps
  created_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP
);

-- ENERGY LOGS
CREATE TABLE public.energy_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id),
  ai_suggested_score INTEGER,             -- what the AI guessed (1-10)
  ai_suggested_level TEXT,                -- 'high', 'balanced', 'low'

```

```

    ai_reasoning TEXT,                -- why the AI suggested this
    confirmed_score INTEGER,          -- what user confirmed (1-10)
    confirmed_level TEXT,             -- final level used by routing
    calendar_event_count INTEGER,     -- how many events were on calendar
    calendar_context TEXT,            -- summary of calendar fed to AI
    logged_at TIMESTAMP DEFAULT NOW()
);

-- SESSION LOGS (thesis data - every task interaction)
CREATE TABLE public.session_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES public.profiles(id),
    task_id UUID REFERENCES public.tasks(id),
    energy_level TEXT,
    energy_score INTEGER,
    was_rerouted BOOLEAN DEFAULT FALSE,
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    ai_copilot_used BOOLEAN DEFAULT FALSE
);

-- COPILOT LOGS (for analytics)
CREATE TABLE public.copilot_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES public.profiles(id),
    message_type TEXT,                -- 'micro_step', 'proactive', 'user_initiated'
    task_id UUID REFERENCES public.tasks(id),
    energy_level_at_time TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- ROW LEVEL SECURITY: users can only see their own data
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.goals ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.energy_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.session_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see own data" ON public.profiles
    FOR ALL USING (auth.uid() = id);

CREATE POLICY "Users see own data" ON public.goals
    FOR ALL USING (auth.uid() = user_id);

CREATE POLICY "Users see own data" ON public.tasks
    FOR ALL USING (auth.uid() = user_id);

CREATE POLICY "Users see own data" ON public.energy_logs

```

```

FOR ALL USING (auth.uid() = user_id);

CREATE POLICY "Users see own data" ON public.session_logs
FOR ALL USING (auth.uid() = user_id);

-- AUTO-CREATE PROFILE ON SIGNUP
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.profiles (id)
    VALUES (NEW.id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

Step 3 — Initialize your project structure

```

mkdir freeside && cd freeside

# Frontend
npx create-next-app@latest frontend \
  --typescript --tailwind --app --src-dir

# Backend
mkdir backend && cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install fastapi uvicorn supabase anthropic \
  google-auth google-auth-oauthlib \
  google-api-python-client python-dotenv

```

Your folder structure:

```

freeside/
├─ frontend/
│   └─ src/
│       └─ app/
│           └─ (auth)/
│               └─ login/page.tsx
│               └─ signup/page.tsx
│           └─ onboarding/
│               └─ page.tsx
│           └─ dashboard/
│               └─ page.tsx
│           └─ layout.tsx
│       └─ components/
│           └─ EnergyPanel.tsx
│           └─ TaskCard.tsx
│           └─ CoPilot.tsx
│           └─ OnboardingWizard.tsx
│       └─ lib/
│           └─ supabase.ts
└─ backend/
    └─ main.py
    └─ routes/
        └─ energy.py
        └─ tasks.py
        └─ copilot.py
        └─ calendar.py
    └─ services/
        └─ router.py      ← cognitive load algorithm
        └─ calendar.py   ← Google Calendar fetcher
        └─ ai.py          ← AI inference + co-pilot

```

Step 4 — Environment variables

```

# frontend/.env.local
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_anon_key
NEXT_PUBLIC_API_URL=http://localhost:8000

# backend/.env
SUPABASE_URL=your_supabase_url
SUPABASE_SERVICE_KEY=your_service_role_key    # from Supabase Settings → API
ANTHROPIC_API_KEY=your_anthropic_key
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret

```

Step 5 — Supabase client setup (Frontend)

```
// frontend/src/lib/supabase.ts
import { createBrowserClient } from '@supabase/ssr'

export const supabase = createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)
```

Step 6 — FastAPI base setup (Backend)

```
# backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from supabase import create_client
import os
from dotenv import load_dotenv

load_dotenv()

app = FastAPI(title="Freeside API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
    allow_credentials=True,
)

supabase = create_client(
    os.getenv("SUPABASE_URL"),
    os.getenv("SUPABASE_SERVICE_KEY")
)

from routes import energy, tasks, copilot, calendar
app.include_router(energy.router, prefix="/energy", tags=["energy"])
app.include_router(tasks.router, prefix="/tasks", tags=["tasks"])
app.include_router(copilot.router, prefix="/copilot", tags=["copilot"])
app.include_router(calendar.router, prefix="/calendar", tags=["calendar"])

@app.get("/health")
def health():
    return {"status": "ok"}
```

PART 1 — Onboarding Flow

Goal: Before the user ever sees the dashboard, Freeside learns who they are, what they want to achieve, and connects the one tool it needs to work intelligently. This data powers every AI decision from day one.

This is 4 screens. Build them as a wizard component with a step counter.

Screen 1 — Welcome & Identity

What the user sees: A calm, dark screen (your brand colors). The Freeside logo. Then:

"Welcome. Before we build your first day, let's understand who you are."

Fields:

- What's your name?
- What best describes you? → [Student] [Professional] [Entrepreneur] [Other]
- In your own words: what does a truly productive day look like for you? (*small textarea, 2-3 sentences is fine*)

Why the last question matters: That free-text answer gets stored in `profiles.productive_day_description` and injected into every AI co-pilot call. It makes the AI's advice feel personal from the very first message.

```
// components/OnboardingWizard.tsx — Step 1
const [step, setStep] = useState(1)
const [profile, setProfile] = useState({
  name: '',
  role: '',
  productive_day_description: '',
})

// On "Continue" → save to Supabase profiles table
const saveStep1 = async () => {
  await supabase
    .from('profiles')
    .update(profile)
    .eq('id', user.id)
  setStep(2)
}
```

Screen 2 — Your Goals

What the user sees:

"What are the 1–3 most important things you want to achieve right now?"

A simple form that adds goal cards. Each goal has:

- Title (e.g., "Launch my side project")
- Category: Work / Personal / Health / Learning
- Timeframe: This month / 3 months / 6 months

Minimum 1 goal required. Maximum 3.

Why this matters: Every AI co-pilot response, every task routing decision, and every proactive intervention is anchored to these goals. When the AI says "Based on your goal to launch your project, I suggest focusing on this task now" — that's powered by this step.

```
// On "Continue" → insert into goals table
const saveStep2 = async () => {
  await supabase
    .from('goals')
    .insert(goals.map(g => ({ ...g, user_id: user.id })))
  setStep(3)
}
```

Screen 3 — Your Work Style

What the user sees: Three quick questions, answered with button selections (not typing):

When do you usually feel most mentally sharp? → ☐ Morning person] [* Afternoon] ☐ Night owl]

How many hours a day do you realistically work? → [Less than 4h] [4–6h] [6–8h] [8h+]

How do you prefer to work? → [Long deep-focus blocks] [Short sprints with breaks] [Flexible, depends on the day]

```
const saveStep3 = async () => {
  await supabase
    .from('profiles')
    .update({
      peak_focus_time: workStyle.peakFocus,
      daily_work_hours: workStyle.hours,
      work_style: workStyle.style,
    })
    .eq('id', user.id)
  setStep(4)
}
```


Why this matters for the algorithm: The routing algorithm uses `peak_focus_time` to boost the priority of deep work tasks during the user's stated peak hours, even when their energy score is only moderate.

Screen 4 — Connect Google Calendar

What the user sees:

"Freeside's superpower is understanding your day before you do. Connect your Google Calendar and we'll analyze your schedule to suggest your energy level each morning — so you never have to guess."

One big button: **[Connect Google Calendar →]**

A smaller link below: *"Skip for now — I'll enter my energy manually"*

Why this is optional: The app works without it. The user can set energy manually. But the calendar connection is what enables the AI inference that makes Freeside genuinely innovative.

Implementation:

```

# backend/routes/calendar.py
from google_auth_oauthlib.flow import Flow
import os

SCOPES = ['https://www.googleapis.com/auth/calendar.readonly']

@router.get("/auth/url")
def get_google_auth_url(user_id: str):
    flow = Flow.from_client_config(
        {
            "web": {
                "client_id": os.getenv("GOOGLE_CLIENT_ID"),
                "client_secret": os.getenv("GOOGLE_CLIENT_SECRET"),
                "redirect_uris": ["http://localhost:8000/calendar/auth/callback"],
                "auth_uri": "https://accounts.google.com/o/oauth2/auth",
                "token_uri": "https://oauth2.googleapis.com/token",
            }
        },
        scopes=SCOPES
    )
    flow.redirect_uri = "http://localhost:8000/calendar/auth/callback"
    auth_url, state = flow.authorization_url(
        access_type='offline',
        state=user_id,
        prompt='consent'
    )
    return {"auth_url": auth_url}

@router.get("/auth/callback")
def google_auth_callback(code: str, state: str):
    # state = user_id
    flow = Flow.from_client_config(..., scopes=SCOPES)
    flow.redirect_uri = "http://localhost:8000/calendar/auth/callback"
    flow.fetch_token(code=code)

    # Store refresh token encrypted in Supabase
    supabase.table("profiles").update({
        "google_refresh_token": flow.credentials.refresh_token,
        "google_calendar_connected": True
    }).eq("id", state).execute()

    # Redirect to dashboard
    return RedirectResponse("http://localhost:3000/dashboard")

```

On the frontend, clicking "Connect Google Calendar" calls GET /calendar/auth/url and redirects to the returned URL.

After this screen, set `onboarding_completed = true` and redirect to dashboard.

PART 2 — Feature 1: Energy Intelligence Engine

The core innovation of Freeside's Bachelor MVP.

Every morning, instead of showing a blank slider and asking "how do you feel?", Freeside fetches the user's calendar, asks Claude to infer their likely energy, and presents a smart suggestion. The user confirms or adjusts in 5 seconds.

Step 1 — Google Calendar fetcher service

```

# backend/services/calendar.py
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials
from google.auth.transport.requests import Request
from datetime import datetime, timezone, timedelta

def get_today_events(refresh_token: str) -> list:
    """Fetch today's Google Calendar events using stored refresh token."""

    creds = Credentials(
        token=None,
        refresh_token=refresh_token,
        token_uri="https://oauth2.googleapis.com/token",
        client_id=os.getenv("GOOGLE_CLIENT_ID"),
        client_secret=os.getenv("GOOGLE_CLIENT_SECRET")
    )
    creds.refresh(Request())

    service = build('calendar', 'v3', credentials=creds)

    now = datetime.now(timezone.utc)
    start_of_day = now.replace(hour=0, minute=0, second=0).isoformat()
    end_of_day = now.replace(hour=23, minute=59, second=59).isoformat()

    events_result = service.events().list(
        calendarId='primary',
        timeMin=start_of_day,
        timeMax=end_of_day,
        singleEvents=True,
        orderBy='startTime'
    ).execute()

    return events_result.get('items', [])

def summarize_events(events: list) -> dict:
    """Turn raw calendar events into structured signals for the AI."""

    summaries = []
    back_to_back_count = 0
    total_meeting_minutes = 0

    for i, event in enumerate(events):
        title = event.get('summary', 'Untitled event')
        start = event.get('start', {}).get('dateTime', '')
        end = event.get('end', {}).get('dateTime', '')

        if start and end:

```

```
start_dt = datetime.fromisoformat(start)
end_dt = datetime.fromisoformat(end)
duration = int((end_dt - start_dt).total_seconds() / 60)
total_meeting_minutes += duration
summaries.append(f"- {title} ({duration} min)")

# Check back-to-back
if i > 0:
    prev_end = events[i-1].get('end', {}).get('dateTime', '')
    if prev_end:
        prev_end_dt = datetime.fromisoformat(prev_end)
        gap = (start_dt - prev_end_dt).total_seconds() / 60
        if gap < 15:
            back_to_back_count += 1

return {
    "event_count": len(events),
    "total_meeting_minutes": total_meeting_minutes,
    "back_to_back_count": back_to_back_count,
    "event_list": "\n".join(summaries) if summaries else "No meetings today"
}
```

Step 2 — AI energy inference service

```

# backend/services/ai.py
import anthropic
import os

client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))

def infer_energy_from_calendar(calendar_summary: dict, user_profile: dict) -> dict:
    """
    Ask Claude to infer the user's likely energy level based on today's schedule.
    Returns: { suggested_score, suggested_level, reasoning }
    """

    prompt = f"""
You are analyzing a user's calendar to estimate their cognitive energy level for today.

USER PROFILE:
- Peak focus time: {user_profile.get('peak_focus_time', 'unknown')}
- Work style: {user_profile.get('work_style', 'unknown')}
- Typical daily hours: {user_profile.get('daily_work_hours', 'unknown')}

TODAY'S CALENDAR:
- Number of meetings/events: {calendar_summary['event_count']}
- Total time in meetings: {calendar_summary['total_meeting_minutes']} minutes
- Back-to-back meetings (less than 15 min gap): {calendar_summary['back_to_back_count']}
- Events:
{calendar_summary['event_list']}

Based on this schedule, estimate the user's available cognitive energy for focused work today.

Consider:
- More meetings = less energy for deep work
- Back-to-back meetings = high mental load
- Heavy meeting days leave little energy for strategic tasks
- Light calendars = more capacity for complex work

Respond ONLY in this exact JSON format, nothing else:
{{
  "suggested_score": <integer 1-10>,
  "suggested_level": "<high|balanced|low>",
  "reasoning": "<one sentence explanation, max 20 words, conversational tone>"
}}
"""

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=150,
        messages=[{"role": "user", "content": prompt}]
    )

```

```
import json
return json.loads(response.content[0].text)
```

Step 3 — Energy inference API endpoint

```

# backend/routes/energy.py
from fastapi import APIRouter, Header
from services.calendar import get_today_events, summarize_events
from services.ai import infer_energy_from_calendar

router = APIRouter()

@router.get("/suggest")
def suggest_energy(user_id: str):
    """
    Called every morning when dashboard loads.
    Fetches calendar → runs AI inference → returns suggestion.
    """
    # Get user profile
    profile = supabase.table("profiles")\
        .select("*").eq("id", user_id).single().execute().data

    if not profile.get("google_calendar_connected"):
        # No calendar connected – skip inference, user will set manually
        return {"mode": "manual", "suggestion": None}

    # Fetch and summarize calendar
    events = get_today_events(profile["google_refresh_token"])
    calendar_summary = summarize_events(events)

    # Run AI inference
    suggestion = infer_energy_from_calendar(calendar_summary, profile)

    return {
        "mode": "ai_suggested",
        "ai_suggested_score": suggestion["suggested_score"],
        "ai_suggested_level": suggestion["suggested_level"],
        "reasoning": suggestion["reasoning"],
        "calendar_event_count": calendar_summary["event_count"],
        "calendar_summary": calendar_summary["event_list"]
    }

@router.post("/confirm")
def confirm_energy(
    user_id: str,
    confirmed_score: int,
    confirmed_level: str,
    ai_suggested_score: int = None,
    ai_suggested_level: str = None,
    ai_reasoning: str = None,
    calendar_event_count: int = None,
    calendar_context: str = None

```



```
):  
    """Called when user confirms or adjusts the energy suggestion."""  
  
    result = supabase.table("energy_logs").insert({  
        "user_id": user_id,  
        "ai_suggested_score": ai_suggested_score,  
        "ai_suggested_level": ai_suggested_level,  
        "ai_reasoning": ai_reasoning,  
        "confirmed_score": confirmed_score,  
        "confirmed_level": confirmed_level,  
        "calendar_event_count": calendar_event_count,  
        "calendar_context": calendar_context,  
    }).execute()  
  
    return {"status": "confirmed", "active_energy_level": confirmed_level}
```

Step 4 — Energy Panel UI (Frontend)

```

// components/EnergyPanel.tsx
'use client'
import { useState, useEffect } from 'react'

export default function EnergyPanel({ userId }: { userId: string }) {
  const [suggestion, setSuggestion] = useState<any>(null)
  const [confirmedScore, setConfirmedScore] = useState<number | null>(null)
  const [isConfirmed, setIsConfirmed] = useState(false)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    // On mount, fetch AI suggestion
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/energy/suggest?user_id=${userId}`)
      .then(r => r.json())
      .then(data => {
        setSuggestion(data)
        if (data.ai_suggested_score) {
          setConfirmedScore(data.ai_suggested_score) // pre-fill slider
        }
        setLoading(false)
      })
  }, [userId])

  const confirmEnergy = async () => {
    const level = confirmedScore >= 7 ? 'high'
      : confirmedScore >= 4 ? 'balanced'
      : 'low'

    await fetch(`${process.env.NEXT_PUBLIC_API_URL}/energy/confirm`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        user_id: userId,
        confirmed_score: confirmedScore,
        confirmed_level: level,
        ai_suggested_score: suggestion?.ai_suggested_score,
        ai_suggested_level: suggestion?.ai_suggested_level,
        ai_reasoning: suggestion?.reasoning,
        calendar_event_count: suggestion?.calendar_event_count,
        calendar_context: suggestion?.calendar_summary,
      })
    })
    setIsConfirmed(true)
  }

  if (loading) return <div>Analyzing your day...</div>

  if (isConfirmed) return (

```

```

<div className="energy-confirmed-banner">
  Energy set: {confirmedScore}/10 – dashboard updated ✓
  <button onClick={() => setIsConfirmed(false)}>Adjust</button>
</div>
)

return (
  <div className="energy-panel">
    {suggestion?.mode === 'ai_suggested' ? (
      <>
        <p className="ai-label">Based on your calendar today:</p>
        <p className="ai-reasoning">"{suggestion.reasoning}"</p>
        <p className="suggestion-score">
          Freeside suggests: <strong>{suggestion.ai_suggested_score}/10</strong>
        </p>
      </>
    ) : (
      <p>How's your energy right now?</p>
    )}

    {/* Slider – pre-filled with AI suggestion if available */}
    <input
      type="range"
      min={1}
      max={10}
      value={confirmedScore || 5}
      onChange={(e) => setConfirmedScore(Number(e.target.value))}
    />

    {/* Visual labels */}
    <div className="energy-labels">
      <span>☐ Low (1-3)</span>
      <span>☐ Balanced (4-6)</span>
      <span>🔥 High (7-10)</span>
    </div>

    <p className="current-score">
      {confirmedScore}/10
      {suggestion?.ai_suggested_score && confirmedScore !== suggestion.ai_suggested_score
        ? ' – adjusted from AI suggestion'
        : ''}
    </p>

    <button onClick={confirmEnergy} className="confirm-btn">
      Start my day →
    </button>
  </div>
)

```

```
)  
}
```

What the user experience looks like: They open the app. They see: *"You have 4 meetings today including a 2-hour strategy session. Freeside thinks your energy for deep work will be moderate (5/10)."* A slider is pre-set to 5. They can drag it or just tap "Start my day →" to confirm. Five seconds. Dashboard updates. Done.

Implementation Steps

Step 1 — Build the task creation form (Frontend)

Create a simple form with:

- Task title (text input)
- Task description (optional textarea)
- Cognitive Load Score — a 1–10 selector with labels:
 - 1–3: Light (admin, quick replies, simple reviews)
 - 4–6: Moderate (meetings, regular work tasks)
 - 7–10: Deep Work (strategic thinking, writing, complex problems)

Do NOT try to auto-detect cognitive load from text yet. Let the user set it manually for v1. This is academically valid — it's called "self-reported task difficulty" and is standard in productivity research.

Step 2 — Build the routing algorithm (Backend)

This is the heart of your thesis. Create a function `route_tasks(user_id, energy_level):`

```

# services/router.py
def route_tasks(tasks: list, energy_level: str, energy_score: int):

    # Define thresholds based on energy
    thresholds = {
        'high':      {'show': range(1, 11), 'priority': range(7, 11)},
        'balanced':  {'show': range(1, 7),  'priority': range(4, 7)},
        'low':       {'show': range(1, 4),   'priority': range(1, 4)},
    }

    config = thresholds[energy_level]

    routed = []
    for task in tasks:
        load = task.cognitive_load_score
        if load in config['show']:
            routed.append({
                **task,
                "priority": "high" if load in config['priority'] else "normal",
                "visible": True
            })
        else:
            routed.append({
                **task,
                "visible": False,
                "reroute_reason": f"Cognitive load {load}/10 is too high for your current energy"
            })

    return sorted(routed, key=lambda x: x['cognitive_load_score'],
                  reverse=(energy_level == 'high'))

```

Step 3 — Create the routing API endpoint

```

@router.get("/tasks/routed")
def get_routed_tasks(user_id: str, db: Session = Depends(get_db)):
    # Get today's latest energy log
    energy_log = db.query(EnergyLog)\
        .filter(EnergyLog.user_id == user_id)\
        .order_by(EnergyLog.logged_at.desc())\
        .first()

    tasks = db.query(Task).filter(Task.user_id == user_id,
                                   Task.status == 'pending').all()

    return route_tasks(tasks, energy_log.energy_level, energy_log.energy_score)

```

Step 4 — Build the task dashboard (Frontend)

Show tasks in two sections:

- **"Your tasks for now"** — visible, routed tasks
- **"Saved for a better moment"** — hidden heavy tasks, shown collapsed with a count: "3 deep work tasks waiting for your high-energy window"

This UI design is critically important for your thesis — it replaces the guilt-inducing "overdue" pile with a calm, intentional rescheduling message.

Step 5 — Log every routing decision

Every time the routing algorithm runs, insert a record into `session_logs`. This gives you quantitative thesis data: how many tasks were rerouted per energy level, average completion rates per energy state, etc.

Feature 3 — AI Co-Pilot with Micro-Step Generation

What it is: A chat assistant that knows the user's energy state, their task list, and their stated goals. Its primary job is to break down a big scary task into 3–5 small, achievable steps when the user asks for help.

What your thesis committee needs to see: A working AI chat that gives advice clearly informed by the user's current context — not generic productivity tips, but specific guidance tied to their energy level and actual tasks.

Implementation Steps

Step 1 — Set up the Anthropic API

```
pip install anthropic
```

```
import anthropic
client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
```

Get your API key at console.anthropic.com.

Step 2 — Build the context builder (this is what makes the AI smart)

Before every AI call, build a context string from the database:

```
def build_context(user_id: str, db: Session) -> str:
    # Get current energy
    energy = db.query(EnergyLog).filter(...).order_by(...).first()

    # Get today's tasks
    tasks = db.query(Task).filter(
        Task.user_id == user_id,
        Task.status == 'pending'
    ).all()
    task_list = "\n".join([f"- {t.title} (load: {t.cognitive_load_score}/10)"
                           for t in tasks])

    # Get user's goals (a simple text field you store on the user profile)
    user = db.query(User).filter(User.id == user_id).first()

    return f"""
You are the Freeside AI Co-Pilot. You are a calm, empathetic productivity coach.

USER CONTEXT:
- Current energy level: {energy.energy_level} ({energy.energy_score}/10)
- Active goals: {user.goals or 'Not set yet'}
- Pending tasks:
{task_list}

BEHAVIOR RULES:
- If energy is low, NEVER suggest heavy work. Always suggest rest or light tasks.
- When asked to break down a task, return exactly 3-5 micro-steps, each under 15 words.
- Be warm and encouraging, never critical.
- Keep responses under 150 words unless breaking down a task.
- Reference their actual tasks and energy level in your responses.
"""
```

Step 3 — Create the chat API endpoint

```
@router.post("/copilot/chat")
def chat(user_id: str, message: str, db: Session = Depends(get_db)):
    system_context = build_context(user_id, db)

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=500,
        system=system_context,
        messages=[{"role": "user", "content": message}]
    )

    return {"reply": response.content[0].text}
```

Step 4 — Build the micro-step breakdown trigger (Frontend)

On every task card, add a small button: **"Break this down"**. When clicked, it automatically sends the message: "Break down this task for me: [task title]" to the co-pilot chat. The response appears either in the chat panel or as an expandable section directly on the task card.

This is a key UX moment — the user never has to think about what to say to the AI. The app does it for them.

Step 5 — Add the proactive energy intervention

When the user sets their energy to "Low", automatically trigger a co-pilot message without the user asking:

```
// Frontend — after energy is set to 'low'
if (energyLevel === 'low') {
  sendToCopilot("My energy is low right now. What should I focus on?");
}
```

This demonstrates proactive AI behavior — one of your thesis's core claims.

Feature 4 — Minimal Gamification & Progress Feedback

What it is: When a user completes a task, they see immediate visual feedback — a satisfying animation, a progress bar update, and a small XP gain. This is the behavioral activation loop in action, and it's enough to prove the psychological principle to your committee without building a full skill tree.

What your thesis committee needs to see: Visual proof that completing tasks feels rewarding. A progress bar or completion counter. Evidence that the system reinforces positive behavior.

Implementation Steps

Step 1 — Add XP scoring logic (Backend)

Simple rule: $\text{XP earned} = \text{cognitive_load_score} \times 10$. A deep work task (load 8) = 80 XP. An easy task (load 2) = 20 XP.

Add an `xp_total` column to your users table.


```

@router.post("/tasks/{task_id}/complete")
def complete_task(task_id: str, user_id: str, db: Session = Depends(get_db)):
    task = db.query(Task).filter(Task.id == task_id).first()
    task.status = 'completed'
    task.completed_at = datetime.now()

    # Award XP
    xp_earned = task.cognitive_load_score * 10
    user = db.query(User).filter(User.id == user_id).first()
    user.xp_total += xp_earned

    # Log the session
    log = SessionLog(user_id=user_id, task_id=task_id,
                    completed_at=datetime.now(), ...)

    db.add(log)
    db.commit()

    return {"xp_earned": xp_earned, "xp_total": user.xp_total}

```

Step 2 — Build the completion animation (Frontend)

When a task is marked complete:

- The task card slides out with a green flash (use Framer Motion: `animate={{ opacity: 0, x: 100 }}`)
- A small toast notification appears: "+80 XP — Deep work complete! 🟩"
- The progress bar at the top of the dashboard fills slightly

This takes less than a day to implement and looks impressive in a live demo.

Step 3 — Build the daily progress bar

At the top of the dashboard, show:

- Tasks completed today: 3/7
- XP earned today: 240
- A circular progress ring showing percentage of today's tasks done

Keep it minimal — one row, three numbers, one ring. No skill trees yet.

Step 4 — Add a weekly summary view

A simple card that shows: "This week you completed 14 tasks and earned 1,240 XP. Your most productive day was Tuesday (High energy, 6 tasks completed)."

This is generated by a simple database query, not AI. It takes 2 hours to build and gives your committee something concrete to look at.

Feature 5 — Behavioral Data Capture & Analytics

What it is: This feature is invisible to the user but it's what makes your thesis academically rigorous. Every action in the app is silently logged so you can produce charts, run analyses, and prove that energy-based task routing actually improves completion behavior.

What your thesis committee needs to see: Graphs. Real data from real users (even if it's just 10–20 test users) showing that tasks completed during high-energy sessions have a higher completion rate and faster completion time than tasks attempted during low-energy sessions.

Implementation Steps

Step 1 — Log everything (Backend)

Every time these events happen, insert a record:

- User sets energy level → `energy_logs`
- Task is viewed/opened → `session_logs` (`started_at`)
- Task is completed → `session_logs` (`completed_at`)
- Task is rerouted by the algorithm → `session_logs` (`was_rerouted` = true)
- AI co-pilot is used → a `copilot_logs` table with message type

Step 2 — Build the analytics query layer

Write these four SQL queries — they become your thesis data:

```

-- Query 1: Average completion time by energy level
SELECT
    energy_level,
    AVG(EXTRACT(EPOCH FROM (completed_at - started_at))/60) as avg_minutes
FROM session_logs
JOIN energy_logs ON session_logs.user_id = energy_logs.user_id
GROUP BY energy_level;

-- Query 2: Completion rate by energy level
SELECT
    energy_level,
    COUNT(CASE WHEN completed_at IS NOT NULL THEN 1 END) * 100.0 / COUNT(*) as completion_rate
FROM session_logs s
JOIN energy_logs e ON s.user_id = e.user_id
GROUP BY energy_level;

-- Query 3: Task rerouting frequency
SELECT
    DATE(logged_at) as date,
    COUNT(CASE WHEN was_rerouted THEN 1 END) as rerouted,
    COUNT(*) as total
FROM session_logs
GROUP BY DATE(logged_at);

-- Query 4: Energy patterns over time (per user)
SELECT
    DATE(logged_at),
    AVG(energy_score) as avg_energy
FROM energy_logs
WHERE user_id = :user_id
GROUP BY DATE(logged_at)
ORDER BY DATE(logged_at);

```

Step 3 — Build the insights dashboard (Frontend)

Create a simple `/insights` page with four charts using the Recharts library (built into React):

- Line chart: user's energy level over the past 2 weeks
- Bar chart: tasks completed per energy level (High / Balanced / Low)
- Bar chart: average time to complete a task by energy level
- Simple stat: "You complete tasks 2.3× faster on high-energy days"

This page is your thesis Figure 1, Figure 2, Figure 3.

Step 4 — Run a small user study

This is the most important step for your thesis grade. Get 10–15 people (classmates, friends) to use Freeside for 2 weeks. Then pull the analytics data and run a basic statistical comparison. Even a simple t-test comparing completion rates on high vs. low energy days gives you quantitative evidence that your system works. Your thesis committee will ask: "Does it actually work?" — this is your answer.

Your Build Order

Do not build these features in parallel. Build them in this exact sequence:

Week 1-2	→	Project setup + Database schema + Auth (Supabase)
Week 3-4	→	Feature 2: Task CRUD + Cognitive Load Routing algorithm
Week 5	→	Feature 1: Energy Input UI + connect to routing
Week 6-7	→	Feature 3: AI Co-Pilot + micro-step generation
Week 8	→	Feature 4: Gamification (XP + completion animation)
Week 9	→	Feature 5: Analytics logging + insights dashboard
Week 10	→	User study (give app to 10-15 people, collect data)
Week 11-12	→	Polish UI, fix bugs, write thesis chapters

What to Tell Your Committee

Your thesis in one sentence: **"Freeside demonstrates that real-time cognitive state estimation, used as input to an adaptive task routing algorithm, produces measurably higher task completion rates and lower self-reported stress compared to static to-do list management."**

Your technical contribution: the routing algorithm + context-aware AI co-pilot. Your psychological contribution: applied Cognitive Load Theory + Behavioral Activation. Your evidence: the analytics data from your 2-week user study. Your future work: LSTM burnout prediction, full external integrations, mobile app.